How PSS, SHAP and LIME Values Are Calculated

Oct 22, 2026 · @Kavin Hiruna

The three charts on the SQLi XAI page answer one question — why did the model give this incident P(attack) = 56%? — but each calculates the answer differently.

1. Payload-Span Shapley (PSS)

PSS measures how much each part of the request text changed the attack probability. For this incident, parentheses / equals signs add about 0.105, quotes about 0.015 and AND / OR logic about 0.

Step 1: Find the evidence groups

This request (`keywords=' or 'a' 'a...`) contains three groups:

- **Q** = quote characters (`'`)
- **L** = AND / OR logic (`or`)
- **S** = parentheses / equals signs (`=`, `(`, `)`)

Step 2: Score the model on every combination

Three groups give $2^3 = 8$ combinations. For each one the engine blanks out the removed groups' characters in the real request, recalculates the 54 features and asks the model for P(attack). Three values are visible on the page:

- Nothing kept (all evidence removed) = 0.44
- Everything kept (the real request) = 0.56
- Q and L kept, S removed (from the counterfactual) = 0.46

Step 3: Apply the Shapley formula

Each group's value is the weighted average of what it adds when it joins each possible set of the other groups:

```
\phi(g) = \sum_{S \subseteq N \setminus \{g\}} \frac{|S|!\,(n-|S|-1)!}
{n!}\,\big[P(S \cup \{g\}) - P(S)\big]
```

With n = 3 groups the weights are: empty set 1/3, one other group 1/6 each, both other groups 1/3.

Worked example

Only the rows marked real come from the page; the other five values are illustrative, chosen to show the method.

Groups kept	P(attack)	Source
none	0.44	real
Q	0.44	illustrative
L	0.44	illustrative
S	0.54	illustrative
Q + L	0.46	real
S + Q	0.56	illustrative
S + L	0.54	illustrative
S + Q + L	0.56	real

For parentheses / equals (S):

S joins	Gain	Weight	Weighted gain
nothing	0.54 − 0.44 = 0.10	1/3	0.033
Q	0.56 − 0.44 = 0.12	1/6	0.020
L	0.54 − 0.44 = 0.10	1/6	0.017
Q + L	0.56 − 0.46 = 0.10	1/3	0.033
Total φ(S)			**0.103**

The same steps give φ(Q) ≈ 0.013 and φ(L) ≈ 0.003. Check: 0.103 + 0.013 + 0.003 = 0.12 = 0.56 − 0.44. This always holds, which is why the page says the values add up exactly. The real chart shows the same pattern (about 0.105, 0.015 and ~0).

The `or`, which a human sees as the most suspicious part, adds almost nothing for this model; the `=` and parentheses push the score up most.

2. SHAP

SHAP uses the same Shapley formula, but over the 54 model features and measured from the average training prediction instead of the evidence-free request.

	Payload-Span Shapley	SHAP (TreeExplainer)
Players	3 groups of request text	all 54 features
Removing a player	blanking real characters in the request	replacing the feature with values seen in training data
Baseline	same request with no SQL evidence (0.44)	average prediction over the training data
Values add up to	P(this request) − 0.44	P(this request) − average prediction

`TreeExplainer` calculates the values exactly by following every decision path in the Random Forest's trees, so it never tries all 2^{54} combinations.

Reading the chart for this incident:

- Positive bars push towards attack: `tcp.dstport` +0.15, `http_get` +0.08, `equals_count` +0.08.
- Negative bars push away from attack: `tcp.srcport` −0.18, `tcp_payload_length` −0.05.

3. LIME

LIME does not use Shapley values; it fits a simple linear model that copies the real model near this one request, and the bars are that model's weights.

1. Make fake neighbours: 2,000 random variations of this request's 54 features, using value ranges from the 300-row background sample.
2. Ask the real model for P(attack) on each variation.
3. Weight the variations by closeness: those near the real request count more.

4. Fit a weighted linear regression: $P \approx w_1 \cdot \text{feature}_1 + w_2 \cdot \text{feature}_2 + \ldots + b$.
5. Show the weights of the 15 strongest features as bars.

The engine uses `discretize_continuous=True`, so each weight means "this feature being in its current range". For example, the `tcp.srcport` bar (−0.25) means that this request's source-port range pushes the score down.

How LIME differs from SHAP:

- The weights do not add up to anything; they are slopes of an approximation, not shares of the probability.
- They vary slightly between runs because the neighbours are random; the engine fixes `random_state=42` so the chart repeats.
- Some weights look odd: `sql_sleep` is +0.08 although the request has no SLEEP, meaning "no SLEEP, in this neighbourhood" goes with a higher P. This is why LIME is used only as a comparison method.

4. What the three charts show together

The model mostly decides this incident from network ports, not from the SQL text: only 0.12 of the 0.56 (about 21%) comes from the payload.

Method	Top reason
Payload-Span Shapley	`=` / parentheses (+0.105)
SHAP	`tcp.dstport` (+0.15), `tcp.srcport` (−0.18)
LIME	`tcp.dstport` (+0.27), `tcp.srcport` (−0.25)

- SHAP and LIME agree that ports dominate the decision.
- `tcp.srcport` is a random number the client chooses for each connection, so it should not decide whether traffic is an attack. The model has probably learned a dataset shortcut, matching the team's TCP source-port ablation results in `outputs/fair_comparison/`.
- PSS confirms the weak payload role, which is exactly the kind of model weakness Component 2 is designed to expose.